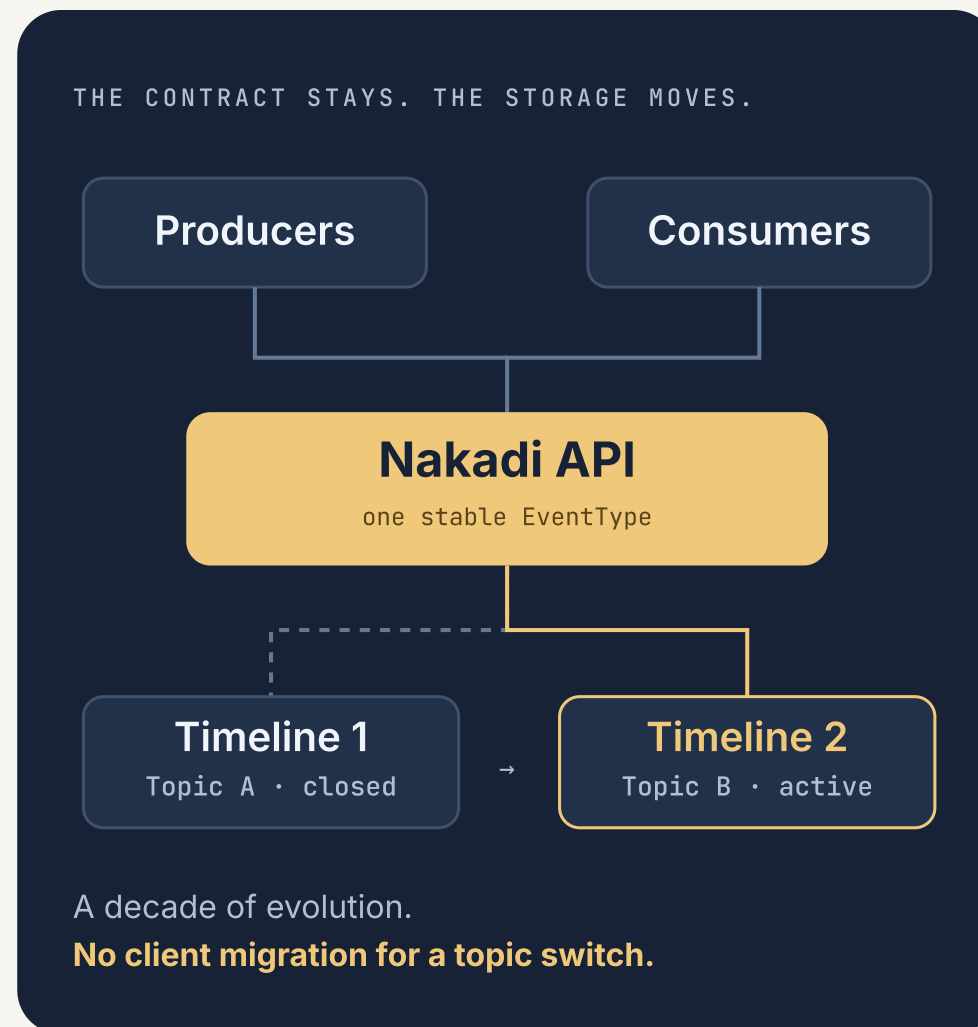


Abstracting Kafka at Scale.

Nakadi's API-first event streaming architecture
at Zalando

Chaitanya Waikar

Senior Software Engineer, Zalando



Who We Are

Europe's leading online fashion and lifestyle platform.

Founded in 2008 · Headquartered in Berlin

62M

Active customers

30

European markets

15000+

Employees

Problems we solve

Fashion logistics at scale: millions of parcels and real-time inventory.

Live personalization for millions of customers.

Resilient, event-driven microservices.

This is where Nakadi lives.

Company figures carried over from the original deck.

BUILD WITH US

We're hiring.

Berlin · Dublin · Dortmund
Helsinki & more

jobs.zalando.com ↗

The Problem

A message broker is a phenomenal primitive.
Handed out directly, it becomes a terrible **contract**.

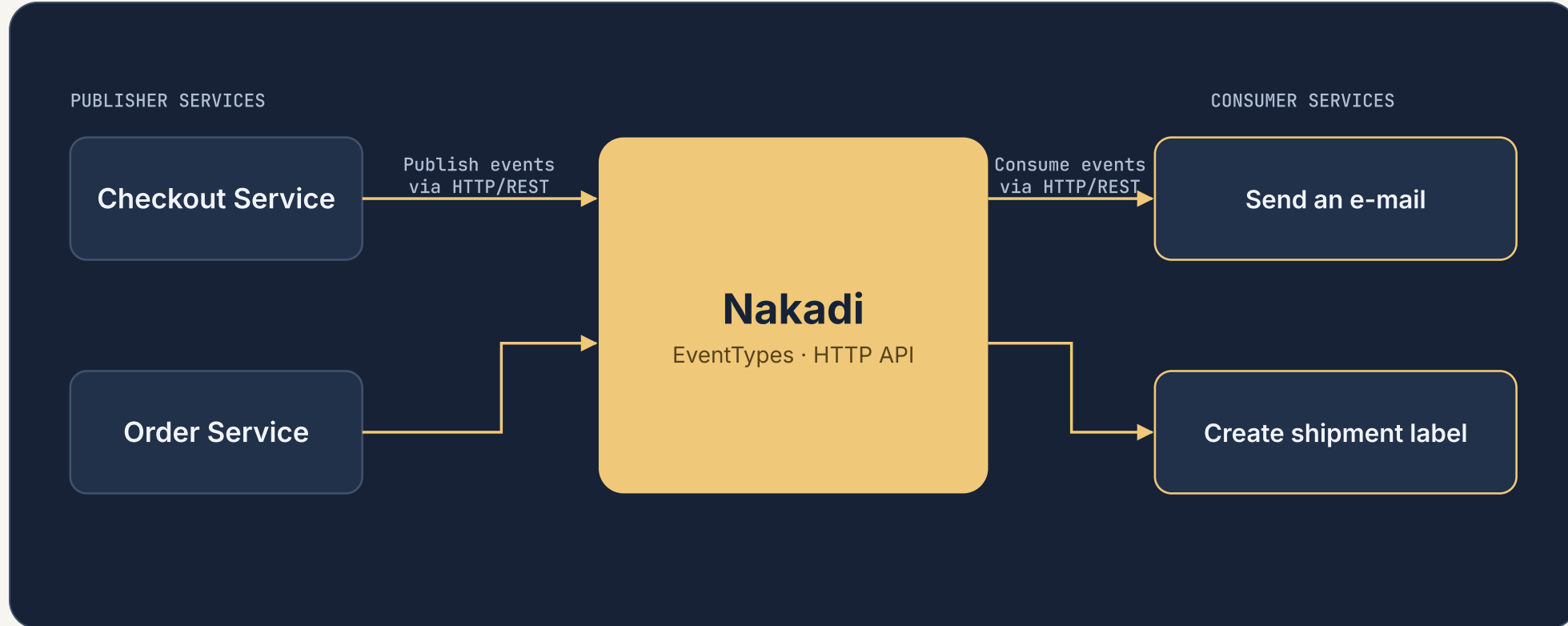
THE SCALE **Hundreds of teams. Thousands of EventTypes.** One shared platform.

THE COUPLING Direct broker access ties clients to **broker lists, physical topics, partition assumptions, and serialization formats.**

THE COST Move a cluster or change the backend, and infrastructure evolution becomes **a coordinated migration across every affected team.**

The real problem isn't moving events. It is **evolving the platform under live traffic without asking every team to change code.**

Nakadi's Position in Zalando



Many independent services. **One shared HTTP contract.** No direct broker access.

What Nakadi Is

A REST abstraction over a **pluggable storage layer**. HTTP in, HTTP out.

An HTTP layer, not a broker SDK.

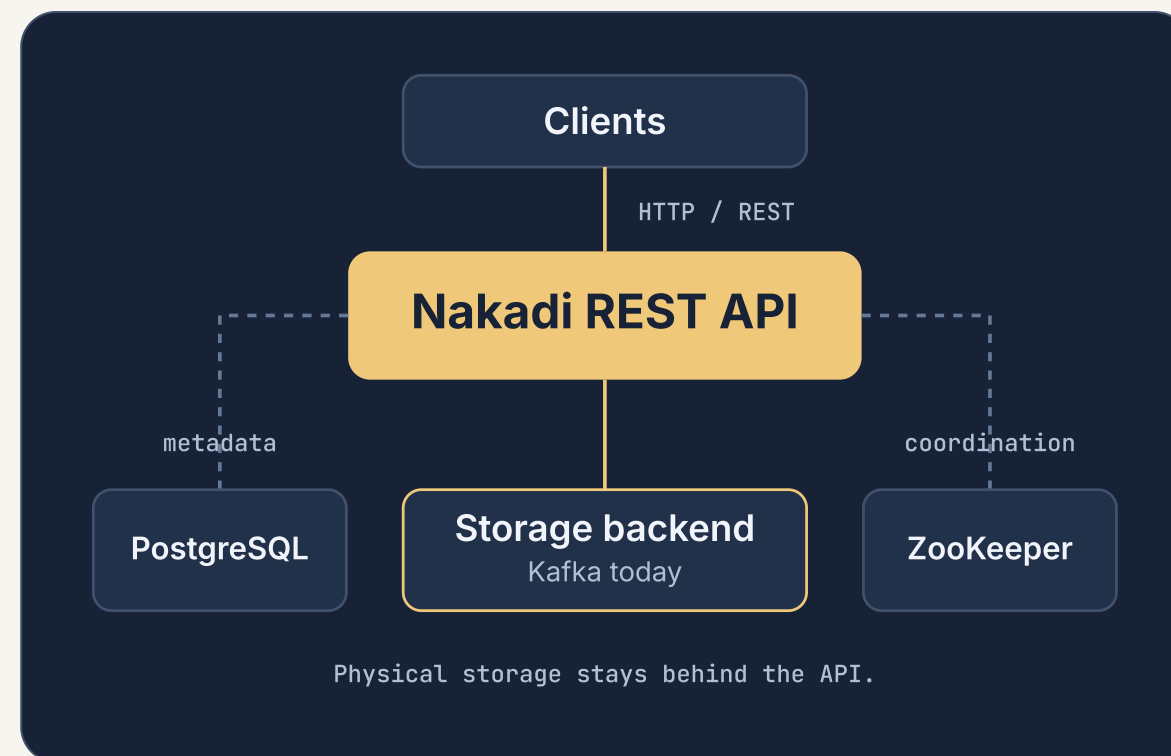
Publish JSON events over HTTP.
Consume batches over an HTTP stream.

The contract is an EventType.

A logical stream with a schema, an owner, partitions, and retention — not a physical topic.

Governed and self-service.

Schema validation and access control live in the platform, not in every client.



Nakadi at Scale

Thousands of contracts. Sustained throughput. Hot individual streams.

7.6k

EventTypes

23.2k

Subscriptions

16k

Parallel connections

~2.5 GB/s

Average cluster throughput

~500k/s

Average cluster event rate

~90

Kafka nodes underneath

2 GB/s

Peak throughput, single EventType

340k/s

Peak publish rate, single EventType

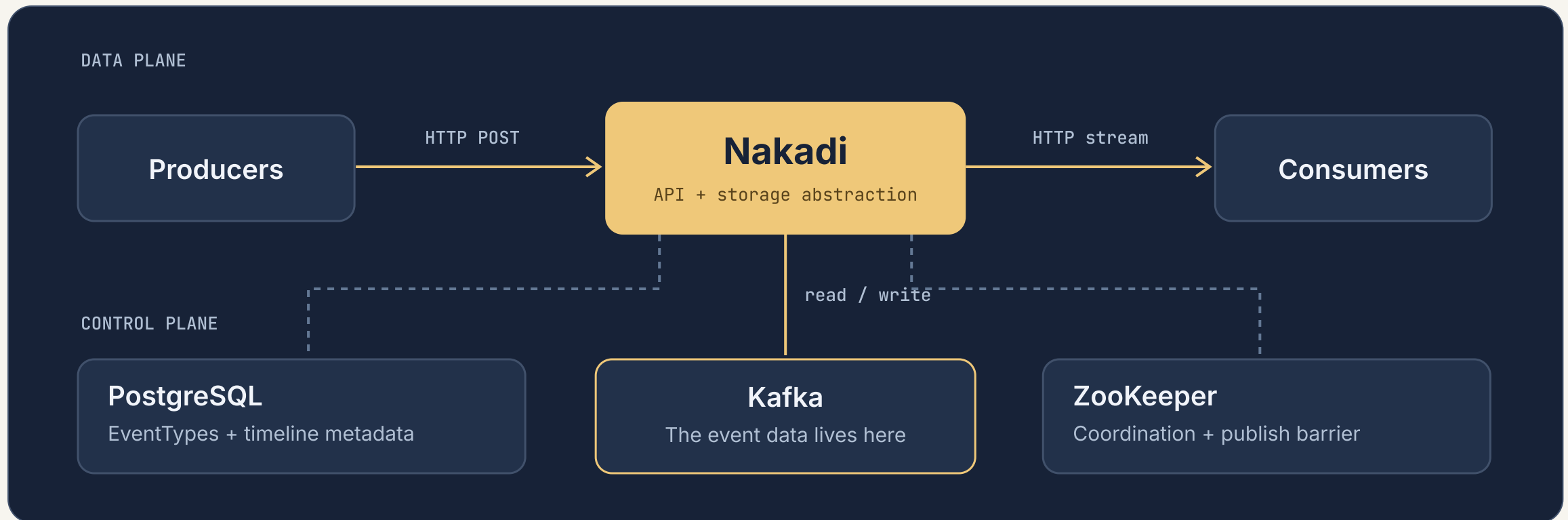
2

Environments: live & staging

All nine figures are from the original deck's scale snapshot; not live telemetry. Cluster averages and per-EventType peaks are distinct.

One API. Two planes.

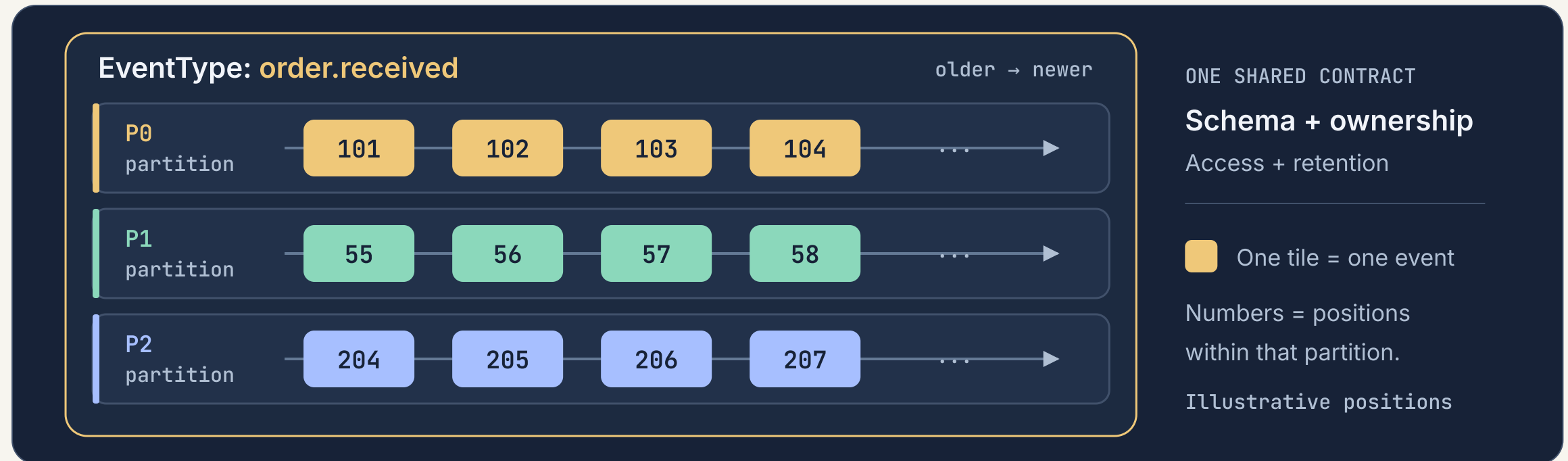
The data plane moves events. The control plane makes change safe.



Kafka is behind the API. **It is not the API.**

One EventType.

A named collection of events, governed as one contract.



Partition to scale.

Distribute events using the configured strategy.

Order is local.

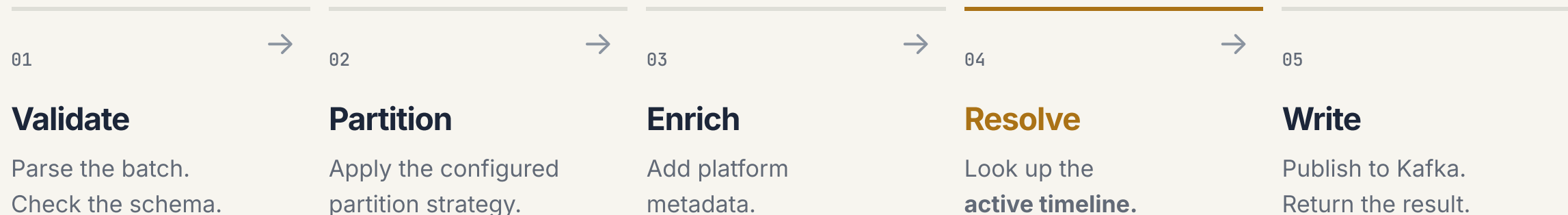
Ordered within a partition, not across partitions.

The name stays logical.

An EventType is not a physical Kafka topic.

A producer sends events. Nakadi chooses the destination.

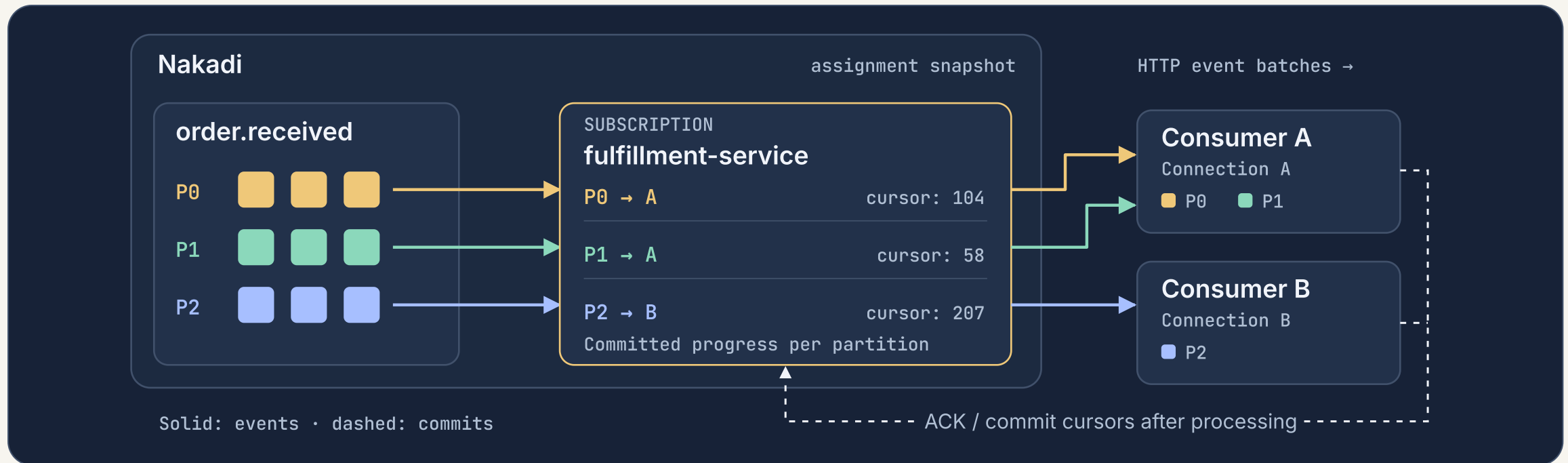
POST /event-types/{name}/events



The indirection that matters: the destination is resolved inside Nakadi, not embedded in the producer.

One subscription. Shared work.

Consumers open HTTP streams. Nakadi assigns partitions and tracks progress.



1/ Connect

GET /subscriptions/{id}/events
Open a long-lived HTTP stream.

2/ Share the partitions

At most one connection per partition
in this subscription. Nakadi rebalances.

3/ Commit progress

POST /subscriptions/{id}/cursors
Resume from the committed cursor.

The Nakadi Ecosystem

One event backbone. Managed services around it.

Nakadi HTTP events EventTypes Subscriptions

Data Lake

Continuously archive EventTypes for analytics and offline processing.

Nakadi SQL

Project, filter, and join event streams with a continuously running SQL query.

Nakadi Connect

Deliver events to databases, object storage, and HTTP endpoints using managed sinks.

Fabric Event Streams

A higher-level platform on Nakadi: declare event pipelines instead of building the plumbing.

Nakadi DLQ(Dead Letter Queue)

Captures unprocessable events via Auto DLQ or an API. Inspect and redrive events to Nakadi using Sunrise UI.

We need to move the stream.

The main cluster is running out of space.

THE READERS

But what about the readers?

Some still need events
in the **old topic**.

THE CLIENTS

And what about the clients?

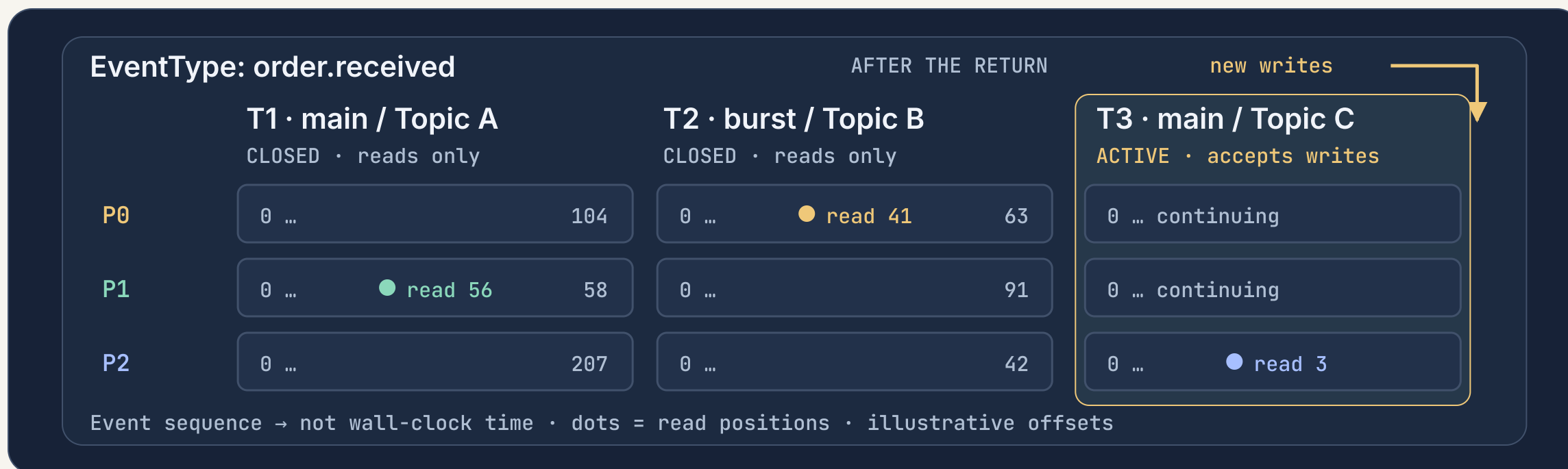
They should keep using
the **same EventType and API**.

The solution: Timelines.

New writes go to the **new topic**.
Readers finish the **old topic**, then follow.

One stream. A history of storage.

Main → burst → main. Each move adds a topic, not a new EventType.



One active write timeline.

New events go to the latest physical topic.

Independent partition progress.

Readers can occupy different retained timelines.

Returning is another move.

T3 uses a new Topic C; it does not reopen T1.

A timeline is a storage mapping.

When T1 closes, record the last published position in each partition.

```
{
  "event_type": "order.received",
  "order": 1,
  "storage_id": "main-cluster",
  "topic": "topic-a",
  "latest_position": {
    "offsets": [104, 58, 207]
  }
}
```

Identity + physical location.

The order sequences the mapping.

The storage and topic locate its events.

Three partitions. Three end positions.

P0

104

P1

58

P2

207

A boundary, not consumer progress.

These ends are fixed when T1 closes.

An empty partition uses the end position -1.

For P0, **104 is the last event in T1**. The next event is in T2.

An offset identifies a location.

Timeline order disambiguates a new topic's offset zero.

PARTS OF A NAKADI OFFSET

001-0002-00000000000000000000

schema version timeline order storage offset

P0: last T1 event

001-0001-00000000000000000104

P0: first T2 event

001-0002-00000000000000000000

A cursor locates the partition, too.

```
{
  "event_type": "order.received",
  "partition": "0",
  "offset": "001-0001-00000000000000000102"
}
```

Illustrative committed cursor: **102**.

The read head may already be at **104**.

Keep offsets opaque.

Clients pass them back; they should not parse them.

Resume after the cursor.

A commit records progress, not the newest event read.

Before the first event.

A storage offset of -1 represents the position before 0.

Finish the old. Follow the next.

After the first switch: P0 is still in T1; P1 has already reached T2.

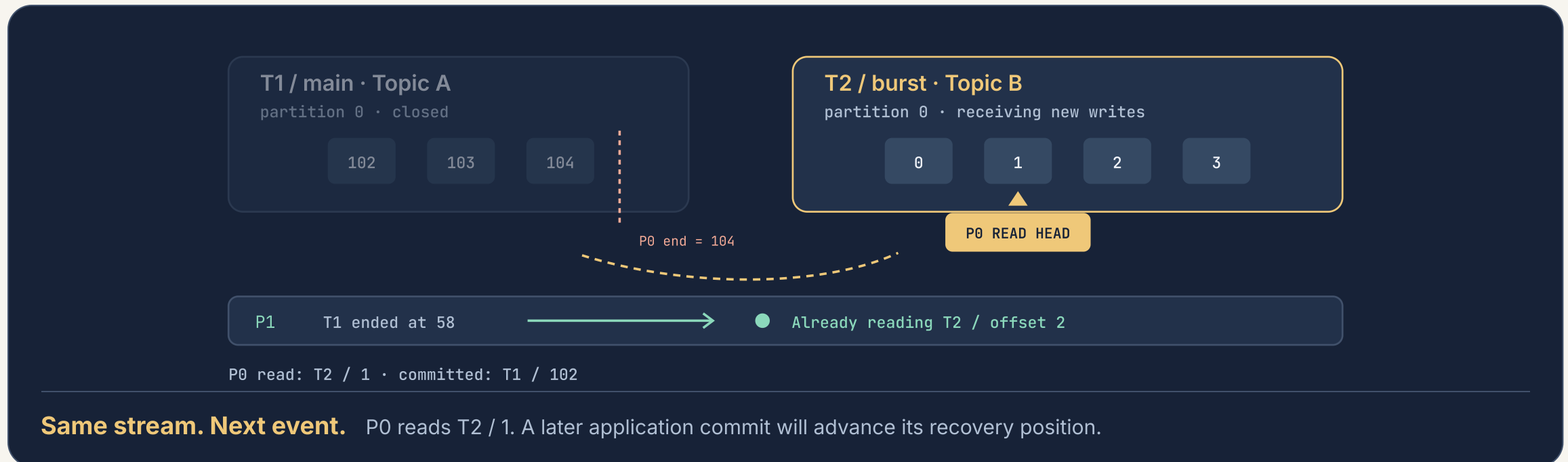
0 / Behind the tail

1 / Read old

2 / Reach boundary

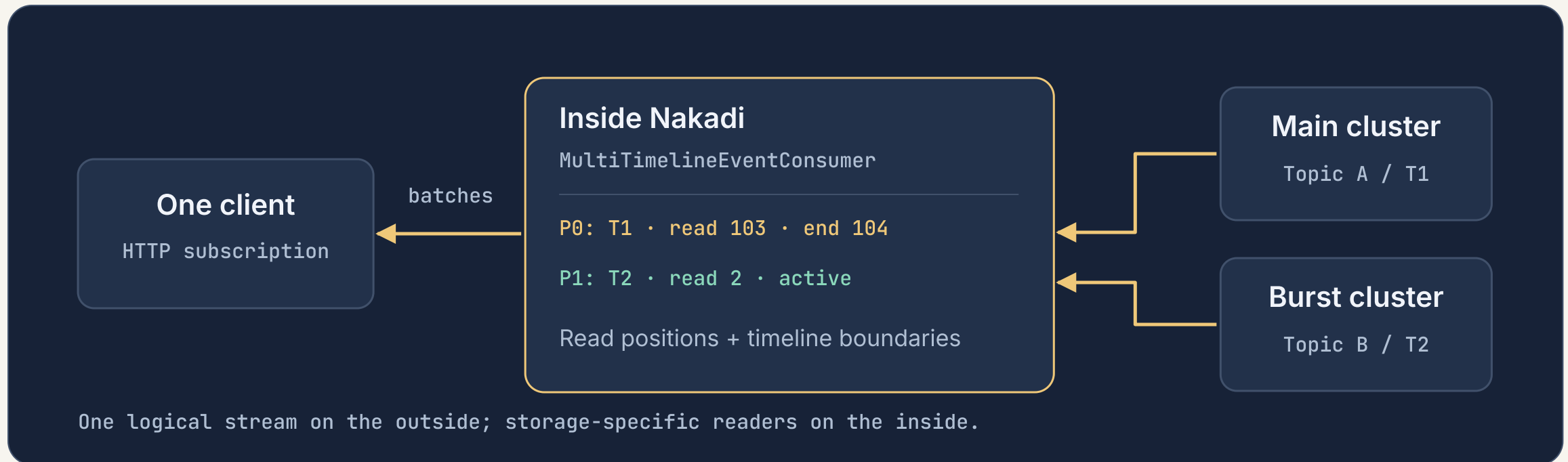
3 / Elect next

4 / Continue



One stream. Multiple storage readers.

Nakadi changes internal readers as individual partitions cross their boundaries.



One admin request. A coordinated cutover.

SELECT THE DESTINATION STORAGE

POST /event-types/order.received/timelines

Content-Type: application/json

```
{  
  "storage_id": "burst-cluster"  
}
```

Admin-controlled

Changing storage is a platform operation.

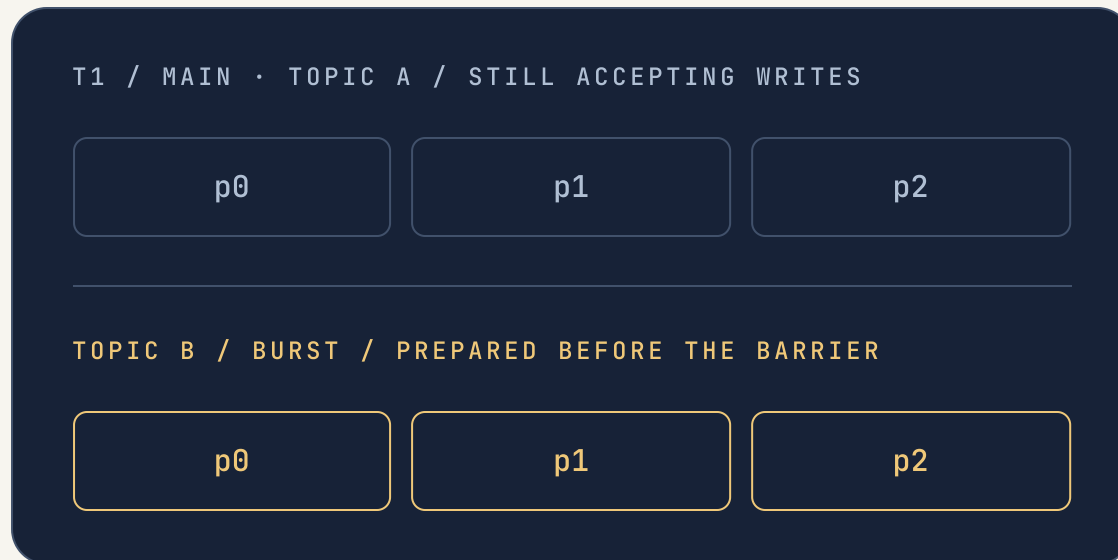
Serialized per EventType

A concurrent switch is rejected, not queued.

Inspectable

GET on the same path lists the timeline chain.

Prepare first. Pause only when you must.



Same partition count.

Provision the destination before blocking publishing.

The active mapping stays put.

A preparation failure does not activate the new timeline.

Stop. Drain. Agree. Switch.

A barrier is not ready until **every node** has finished its in-flight writes.

0 / Publishing

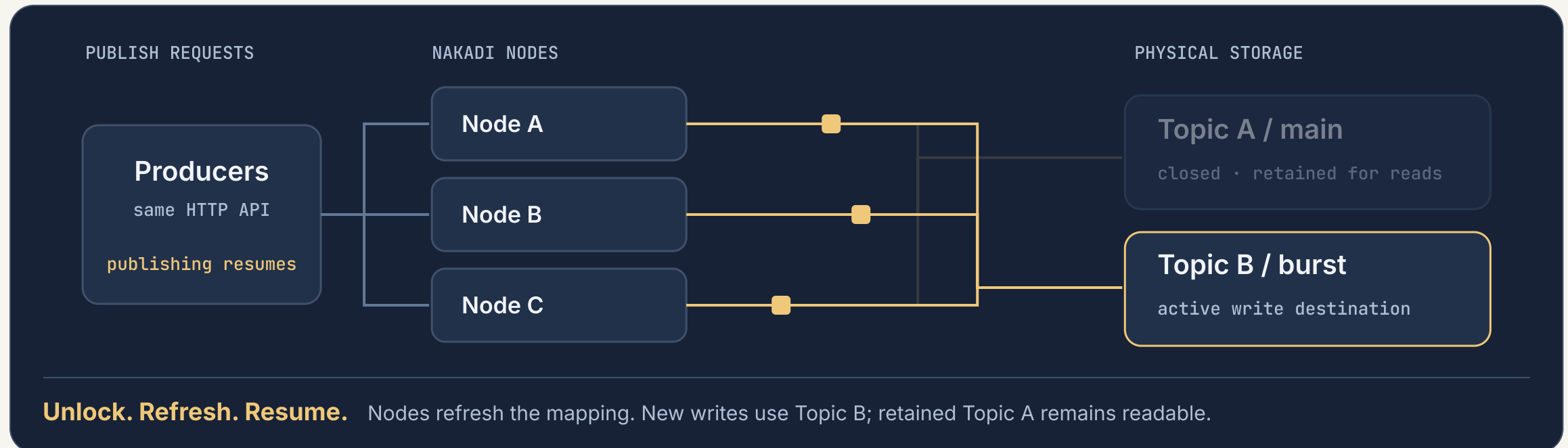
1 / Lock

2 / Drain

3 / All ACK

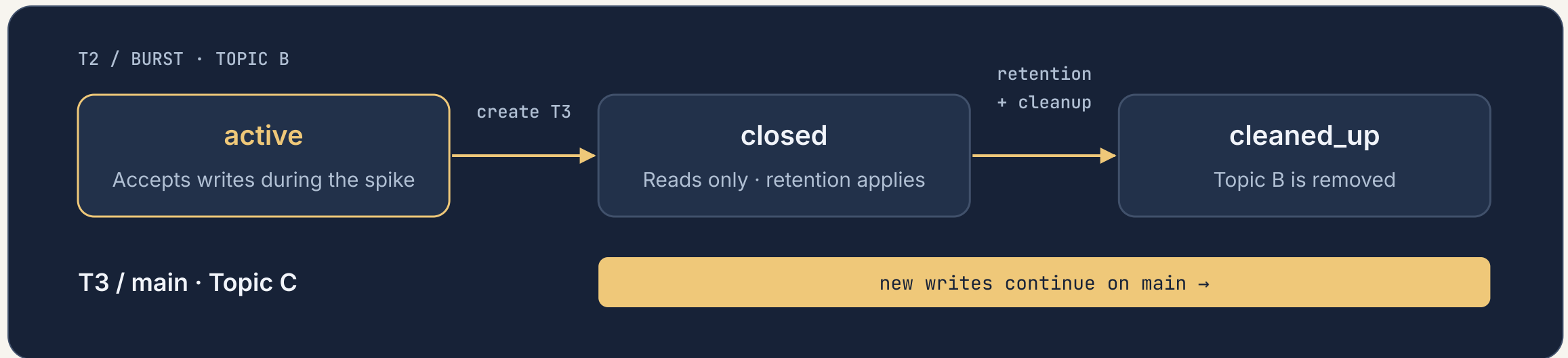
4 / Commit

5 / Resume



Move back. Retire the burst cluster.

The return creates T3 on main / Topic C. It does not reopen T1.



Closed does not mean deleted.
Readers may still need retained events in T2.

Retire the cluster last.
Wait for retained data to expire and topic cleanup to finish.

Failure handling is part of the architecture.

FAILURE	WHAT THE SWITCH DOES	WHAT TO REMEMBER
Preparation fails	The old timeline remains active.	NO MAPPING CHANGE
Barrier times out	Abort the attempt; roll back the lock.	DO NOT CUT OVER EARLY
Metadata commit fails	Roll back metadata; attempt unlock.	NEW TOPIC CAN BE ORPHANED
A second switch races	Reject the concurrent operation.	ONE SWITCH PER EVENTTYPE

Known gap: a prepared topic is not automatically removed after every later failure.

No client migration.

Not zero coordination cost.

A per-EventType publish pause

New writes wait or can time out while the barrier is held.
Retry behavior still matters.

Coordination is a dependency

ZooKeeper health matters for the barrier and state propagation.

A storage move, not repartitioning

The switch preserves partition count. Increasing it uses a separate partition-count API.

An operator-controlled scope

Admins select storage and own cleanup. Compacted EventTypes cannot be timed.

“Transparent” describes a stable API and automatic routing, not an unconditional availability guarantee.

Stable contract. Replaceable infrastructure.

01 / DECOUPLE

Name the logical thing.

Keep physical placement behind an API.

02 / COORDINATE

Stop and drain.

Account for work that already started.

03 / RECORD

Commit the boundary.

Make the old end and new home durable.

04 / CONTINUE

Let readers follow.

Cross the seam in order, within retention.

The hard part is not creating a new topic.

It is making the change safe for everyone already using the old one.

— QUESTIONS / 15 MINUTES

Change the storage. Keep the promise.

Timelines move the physical stream forward.
Nakadi keeps producers and consumers on the same logical one.

Chaitanya Waikar

Senior Software Engineer, Zalando